# A Comparison of Debugging Features in JetBrains' IntelliJ IDEA and the Eclipse Foundation's Eclipse IDE

## Independent Technical Analysis

**By: Sajid Awadalla, Aesha Shah, Ansh Shah, Indira Tatikola**

### Table of Contents

1. Executive Summary
2. User Interface
3. Software Design & Architecture
4. Algorithms
5. APIs
6. Source Code
7. Intellectual Property Infringement Risk Assessment
8. References

## Executive Summary

This report presents an independent technical analysis comparing the debugging capabilities of JetBrains' IntelliJ IDEA and Eclipse Foundation's Eclipse IDE to assess potential intellectual property overlap.

Through comprehensive examination of user interfaces, software design and architectures, algorithms, APIs, and source code implementations in tandem with review of relevant case law, our analysis reveals that IntelliJ IDEA and Eclipse IDE have fundamentally different approaches in debugging functionality that minimize infringement risk. The similarities identified, throughout the report, are consistent with industry standards and necessary functionality in software debugging, rather than with protected intellectual property.

## User Interface

This section discusses how the user interfaces (UIs) of IntelliJ IDEA and Eclipse IDE are designed to help developers find and fix programming errors. The user interface is what the user actually sees and interacts with on the screen of the application. Generally, it includes the layout of buttons, menus, color highlights, and information panels. In a programming environment for debugging, the UI plays a major role in how easily a developer can identify a mistake in the code and understand how to fix it.

**IntelliJ IDEA**

Within IntelliJ IDEA, everything the programmer needs for debugging is located in a single, organized workspace. The code editor (where the developer writes code), the project explorer (which lists all files), and the output console (which shows program results and errors) all appear in the same window.

When a problem with the code is detected (sometimes referred to as a “bug” in the code, hence the term “de[bug]ger”), the software automatically highlights the specific line with a visual indicator (our team has included visuals of some identified indicators) that something is wrong and/or should be modified.

If the user hovers the mouse over the indicator, it displays a short description of the problem, such as expected syntax missing, unresolved method, or redundant call.

Additionally, the description is followed by suggested fixes / issue resolutions. These provide multiple solutions of corrected code following Java’s syntax and logic rules, which ensures that when programmers “accept” the code modification (e.g. clicking `Import static method...` from Figure 1b) it will not cause repeated issues.

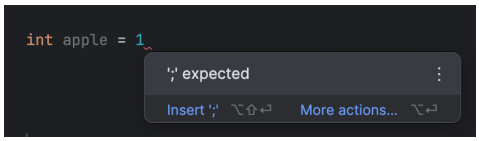



Figure 1a. red wavy/squiggly line

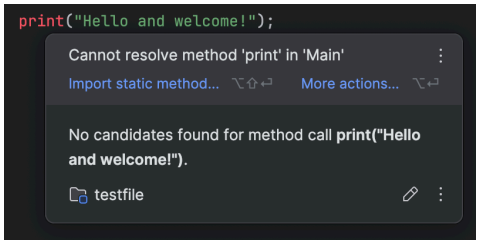



Figure 1b. red text color

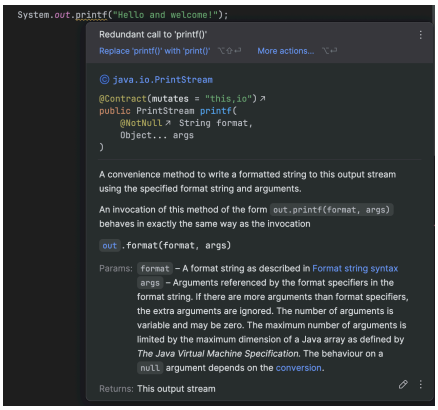



Figure 1c. yellow wavy/squiggly line (not always an issue / bug, usually just a suggestion for optimization or clarity)

The user can also navigate to all the bugs in the file individually by clicking on the colored line in the scroll bar (usually on the top right of the code file) or the `Problems` tab (usually in the bottom left panel).

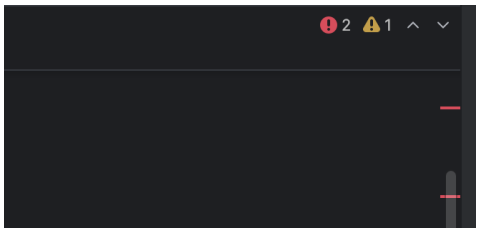



Figure 2a. symbols located in the top right section of code file, alerting to multiple errors / suggested modifications

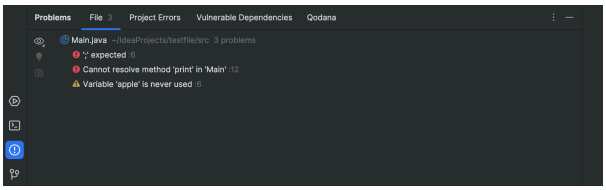



Figure 2b. `Problems` tab located in the bottom left panel, providing a list of errors / suggested modifications

**Eclipse IDE**

Eclipse IDE’s user interface performs the same fundamental task as IntelliJ IDEA’s UI, but it does so within a modular, multi-pane workspace that emphasizes extensibility over a consolidated layout.

The initial UI is similar to that of IntelliJ IDEA, underlining the part of the code that contains the error, and explaining the issue.

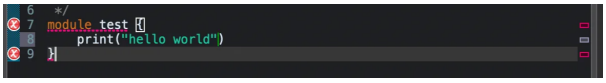



Figure 3a. red wavy/squiggly line, `X` indicators on the left of each line, and red rectangular indicators on the right of each line

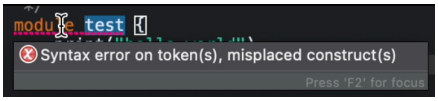



Figure 3b. description of error

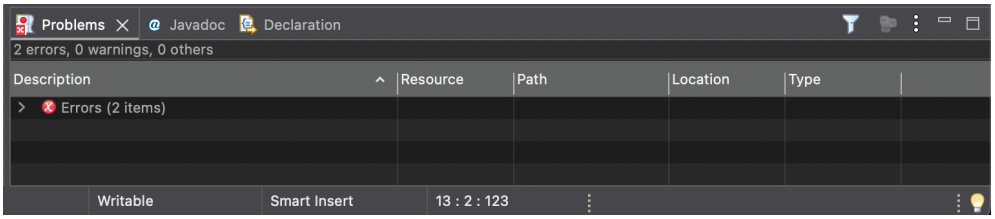



Figure 3c. `Problems` tab

However, it provides solutions across multiple tabs and indexes. To find the immediate fix to an error, the user must right-click to locate a menu of options. Selecting “Quick Fix” would provide the correct syntax and resolve the error.

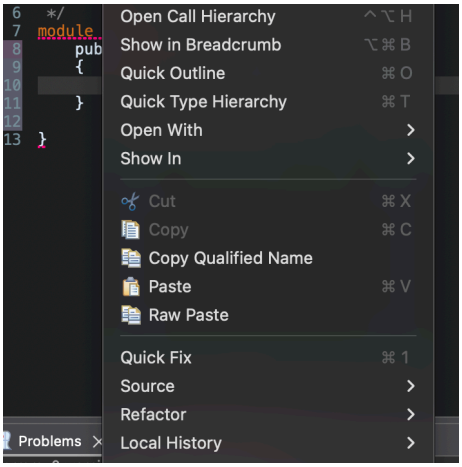



Figure 3d. drop-down menu

## Software Design & Architecture

From a software architecture perspective, the Eclipse IDE and IntelliJ IDEA debuggers represent fundamentally different approaches to solving the same core problem: enabling developers to inspect and control running programs.

Below are a series of figures (created by our team) demonstrating the command flows and data flows within the software architectures of IntelliJ IDEA’s and Eclipse IDE’s debugger components respectively.

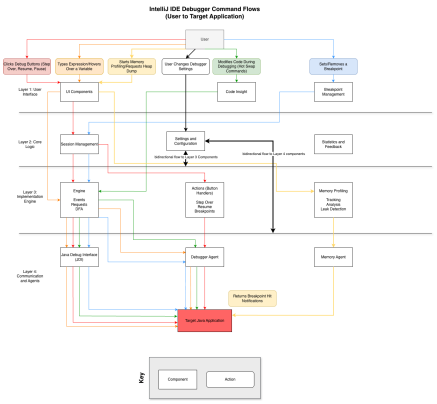



Figure 4a. IntelliJ IDEA Debugger Command Flows (From User to Target Application)

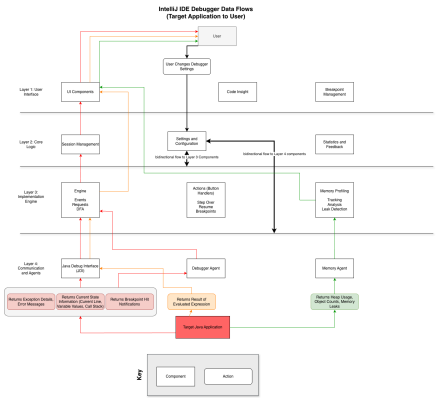



Figure 4b. IntelliJ IDEA Debugger Data Flows (From Target Application to User)

IntelliJ IDEA implements a tightly-integrated, layered architecture optimized specifically for Java development. Rather than supporting arbitrary extensions, IntelliJ's debugger consists of four layers with predetermined, static relationships: frontend user interface, backend coordination logic, implementation engine, and communication agents. This creates what software engineers call "early-binding" — all the connections between components are established at build time, resulting in faster execution but less flexibility. IntelliJ IDEA's approach resembles a high-end appliance designed for one specific task extremely well, with innovations centered on agent injection architecture where specialized debugging helper programs are dynamically inserted into target applications, sophisticated hot-swapping (changing code in a running program without stopping it) implementation using Java Virtual Machine (JVM) Tool Interface callbacks, and tightly-coupled component communication through direct (i.e. without using an intermediary system) method calls.

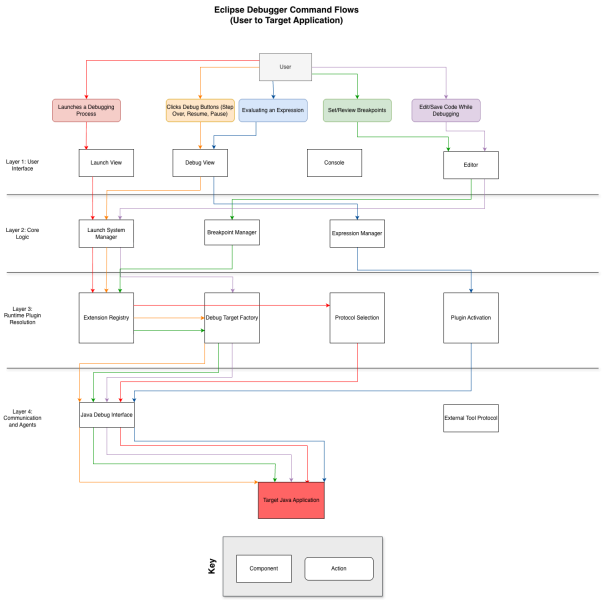



Figure 5a. Eclipse IDE Debugger Command Flows (From User to Target Application)

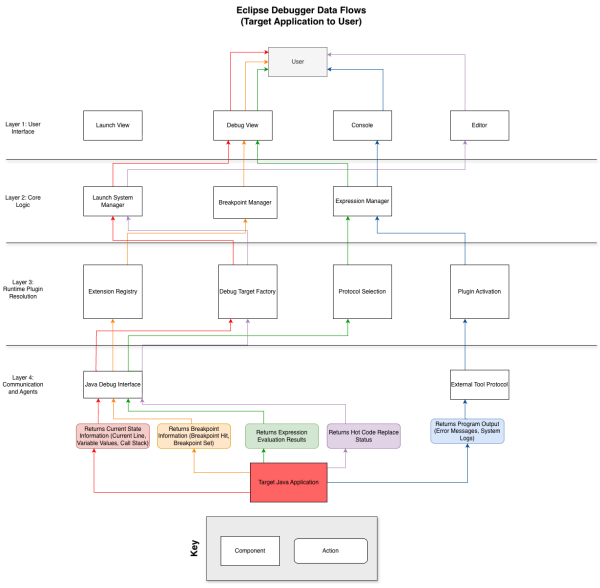



Figure 5b. Eclipse IDE Debugger Data Flows (From Target Application to User)

Eclipse IDE, in contrast, employs a modular, extensible framework architecture built around what software engineers call "extension points" — essentially predetermined hooks where third-party developers can “plug-in” additional functionality. This is much like a universal power tool system where the base unit provides the motor and control electronics, but different attachments can be added for specific tasks. Eclipse IDE's debugger consists of five distinct layers: a user interface framework, core debugging logic, a runtime plugin resolution system, communication protocols, and target processes. The key innovation here is Layer 3: the runtime plugin resolution system, which dynamically determines at execution time which specific debugging implementation should handle a given task. This creates a "late-binding" architecture where decisions about how to debug are made when needed, not when the software is compiled. Eclipse IDE resembles a modular platform designed to accommodate many different tasks adequately.

## Algorithms

IntelliJ IDEA’s and Eclipse IDE’s debuggers approach their internal procedure for debugging in different ways. To better explain the algorithms that guide the developer’s debugging experience, it is important to understand how each IDE pauses execution, steps through the running program, and reveals runtime states.

### *Breakpoint*

A breakpoint is a “flag” the user can set on a line of code to indicate to the debugger it needs to stop the program at that specific line of code to allow the user to further analyze it.

When a breakpoint is encountered in IntelliJ IDEA, the execution of the program is temporarily halted and a snapshot is constructed, taking into account factors such as which method the line of code falls within and which local variables exist (and their corresponding currently assigned values). Then, IntelliJ IDEA walks through that code fragment step by step. It tests a few lines safely using the values from that paused state, without actually resuming normal execution.

On the other hand, Eclipse IDE has a segmented structure, as its debugging model is built in such a way so that multiple languages can connect to and use the same underlying framework, not just Java (whereas IntelliJ IDEA is primarily for Java). Instead of relying on one procedure for evaluating expressions, the Eclipse IDE system sends a general request to whichever component is controlling the current language selection in the programming session. The component is called a “plug-in,” which is an add-on that knows how to work with the specific programming language and can be swapped in or out without changing the core system. It acts as a dispatcher in this way by pausing the code and handing control to the active language plug-in before finally returning the results back to the IDE.

### *Stepping*

Stepping means running the program one small line, or block, at a time, instead of all at once. Both IDEs offer the ability to step into, over, and out of code. A similar distinction, as the breakpoint functionality, is seen in stepping.

Intellij IDEA’s debugger provides the user with extra context like asynchronous call tracing (keeps track of the sequence of events a program follows when certain tasks run in the background) and memory inspection. With asynchronous call tracing, IntelliJ IDEA keeps track of the chain of actions a program takes. In Java, parts of the program can run simultaneously on the side while the rest keeps going, which might cause confusion about the steps taken thus far. This feature reconnects those jumps and shows a clear story for the user to better understand where their code is in execution. With the power of memory inspection, users are able to get a better grasp on how much memory a certain aspect of a program is holding onto.

Eclipse IDE handles stepping differently in a more language-neutral manner by sending a standardized request to the program to run just enough instructions to complete the next step, then pause again and report where execution stopped. It is much less nuanced but aligns with the broader goal of supporting many programming languages. Eclipse IDE's stepping process is more flexible across languages, particularly because it allows enhancements by language-specific plug-ins.

## APIs

Both Eclipse UDE and IntelliJ IDEA have systems, called APIs (Application Programming Interfaces), that allow the IDE to handle debugging features such as breakpoints, stepping, variable inspection, and expression evaluation. These APIs are also what plugin developers use when they want to extend or customize how debugging works inside the IDE.

While they share similar goals, the internal organization and naming conventions differ. Eclipse IDE keeps nearly all debugging-related components inside one unified namespace called `org.eclipse.debug.*`, while IntelliJ IDEA spreads its debugger across several separate namespaces, mainly `com.intellij.debugger.*` for the older Java-specific system and the newer version, `com.intellij.xdebugger.*` which is a flexible and language-agnostic.

In Eclipse IDE, the debugging system is divided into several defined layers, which makes it easier for someone to trace how different parts fit together. The lowest layer is the core system, located in `core`, which is responsible for managing debug launches, breakpoints, threads, and debug targets. Alongside this is the model layer, found in `core.model`, which defines interfaces used throughout the entire debugging system. These include `IDebugTarget` (representing the program being debugged), `IThread` (representing a path of execution), and `IBreakpoint` (representing a location where the program pauses). On top of these layers, Eclipse IDE provides a user-interface layer through `debug.ui`, which provides the visual components for the `Debug` view, displays variables, and defines actions that users trigger when stepping through code or evaluating expressions. Eclipse IDE further separates the process of starting programs into the `debug.core.launch` package, which controls how debug sessions are initiated and configured. To help the debugger connect running programs to their source files, Eclipse IDE also includes the `debug.core.sourcelookup` package, which is responsible for mapping stack traces and runtime information back to the correct lines of source code. Additional support for expression evaluation and variable display is found within UI-related packages such as `debug.ui.actions` and `debug.ui.sourcelookup`. Eclipse IDE's debugging API is vertically integrated; each package has a clearly defined purpose.

IntelliJ IDEA organizes its debugging system very differently. Instead of placing everything under one location, IntelliJ's debugging features are spread across several namespaces, reflecting the IDE's evolution over time and its support for many programming languages. The older Java-specific debugger lives primarily inside `com.intellij.debugger` and `com.intellij.debugger.engine`, which oversee debugger sessions, communication with the Java Virtual Machine (JVM), and the general control of the debug process. IntelliJ IDEA also includes `com.intellij.debugger.jdi`, a set of wrappers around the Java Debug Interface (JDI), the low-level mechanism used to communicate with threads, variables, and runtime values inside the JVM. The user interface for IntelliJ's debugging system, including the debugger tool window, breakpoint panels, and variable inspectors, are defined in packages like `com.intellij.debugger.ui` and `com.intellij.debugger.ui.breakpoints`, while the actions that perform stepping, resuming, and expression evaluation are defined inside `com.intellij.debugger.actions`.

Later in its development, IntelliJ IDEA introduced a new, more modular and language-agnostic debugging framework known as the `XDebugger`, which is in the `com.intellij.xdebugger.*` packages. This framework was designed so that all JetBrains IDEs (such as PyCharm for Python, CLion for C/C++, and Rider for .NET) could share a common debugging infrastructure. As a result, the `XDebugger` system is broken into many focused components: `xdebugger.frame` manages stack frames and the representation of variables, `xdebugger.breakpoints` handles different breakpoint types, and `xdebugger.evaluation` defines how expressions are evaluated during a

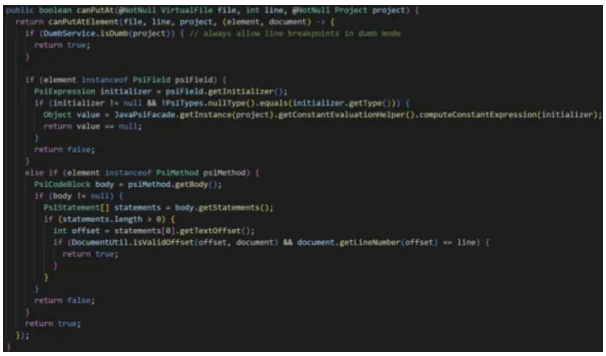
debug session. IntelliJ IDEA also provides `com.intellij.xdebugger.attach` for attaching to running processes, `com.intellij.xdebugger.settings` for storing debugger configuration, and a runtime Java agent located in `com.intellij.rt.debugger.agent`, which collects debugging information from target JVMs. IntelliJ IDEA integrates debugging with its overall run and execution system using the broader `com.intellij.execution.*` packages, which connect debugger sessions with run configurations, consoles, and process control.

The main difference is how each API describes the debugging components. Eclipse IDE focuses mostly on defining small, specific interfaces (such as `IDebugTarget`, `IThread`, and `IBreakpoint`) that represent individual pieces of a debugging session. IntelliJ's `XDebugger` instead defines higher-level abstractions (like `XDebugSession`, `XDebugProcess`, and `XBreakpointType`) which describe entire debugging workflow, not just isolated components. Also, the two IDEs have differences in their extensibility models. Eclipse IDE has interface inheritance and plugin definitions to extend debugging features, while IntelliJ IDEA uses an extension-point system that is more event-driven and modular. Eclipse IDE offers a cohesive debugging API that is simple to follow, whereas IntelliJ's design is more flexible with multiple layers and packages.

## Source Code

To compare the source code of IntelliJ IDEA and Eclipse IDE, it is useful to narrow the scope to common functionality, a core feature of their debuggers: *line breakpoints*. A line breakpoint lets the debugger know exactly where it should pause during a program's execution so the user can examine what is happening at that moment.

The line breakpoint code in IntelliJ IDEA is direct and focused on the immediate task. The `JavaLineBreakpointType` implementation lays out the logic for determining whether a breakpoint should trigger in a clear, step-by-step sequence. A reader can easily follow how IntelliJ IDEA decides to pause on a particular line without needing to understand a larger underlying framework.



This small method simply answers "Can we stop here?" with a few direct checks and conditionals, a style that is easy to read and tightly focused on Java's workings

In Eclipse IDE, the code behind line breakpoints reflects a framework-centered and highly standardized style. The `LineBreakpoint` class is structured around a common pattern in which values are stored in a shared data structure and accessed through carefully-defined getter methods. Variable names follow consistent conventions, and the implementation leans heavily on reusability by relying on interfaces from the overarching Eclipse IDE platform. The result is code that is predictable, consistent, and clearly intended to integrate smoothly with other parts of the system.




These small “getter” methods all follow the exact same template: same naming pattern and error handling. This ensures other Eclipse IDE parts can plug-in, a clear example of a standardized and integration-friendly style

## Intellectual Property Infringement Risk Assessment

After comparing IntelliJ IDEA and Eclipse IDE across the UI, architecture, algorithms, APIs, and source code, our overall conclusion is that the risk of intellectual property infringement between the two IDEs is low. Even though both tools provide similar debugging features (because debugging is fundamentally a standardized activity), the actual ways they implement these features are noticeably different. To ground our analysis, we looked at several major software-related court cases to understand how similarity is judged in real legal situations.

Starting with the user interface, IntelliJ IDEA and Eclipse IDE both use error indicators, breakpoint markers, and panels for inspecting variables, but they organize them very differently. IntelliJ keeps everything in a unified workspace, whereas Eclipse provides a modular, multi-pane environment that can be rearranged. This kind of similarity is expected in debugging tools, and *Apple Inc. v. Samsung Electronics Co.* (2012) helps explain why. In that case, Apple argued that Samsung copied the “look and feel” of the iPhone interface. The court had to separate functional necessities (things every smartphone must have to work) from protectable design choices. Similarly, *Lotus Dev. Corp. v. Borland Int’l, Inc.* (1996) established that menu structures and command hierarchies are often unprotectable because they act as “methods of operation.” In this IDE case, the overlapping UI elements between IntelliJ IDEA and Eclipse IDE fall into the functional category. They are not unique artistic expressions but typical debugging mechanics every developer expects.

We also considered *Microsoft Corp. v. i4i Ltd. Partnership* (2011), which dealt with the validity of a patent based on document-editing features in Microsoft Word. The key point from that case is that a UI feature must represent a distinct, novel innovation to be protectable. IntelliJ’s more polished UI elements (like real-time automatic fix suggestions) could be seen as advanced features, but Eclipse doesn’t implement them in the same way, nor does IntelliJ’s version resemble anything Eclipse has built. There is no overlap in the unique, “patent-like” aspects of each UI, which supports a low infringement risk.

In software architecture, IntelliJ and Eclipse differ even more clearly. IntelliJ uses a tightly integrated, early-binding structure that focuses mainly on Java, while Eclipse uses a plugin-based, late-binding framework built for extensibility. The reasoning in *Johnson Controls, Inc. v. Phoenix Control Systems, Inc.* (1989) is especially relevant here. The court in that case emphasized evaluating the structure, sequence, and organization (SSO) of software, not just the final output. When we look at IntelliJ and Eclipse through that lens, they are not similar at the architectural level. Their command flows, data flows, and component

interactions are designed very differently. Even when they both support breakpoints or stepping, the pathways they take to accomplish those tasks are distinct.

Algorithmically, IntelliJ IDEA dives deeper into Java-specific features, using things like JVM TI callbacks, asynchronous call tracing, and agent injection. Eclipse IDE instead uses a generalized system where the active language plugin decides how to evaluate breakpoints or step through code. This is again consistent with J*ohnson Controls, Inc. v. Phoenix Control Systems, Inc.* (1989), where the court found infringement only when the internal process or "flow of the program" was substantially the same. Here, IntelliJ and Eclipse clearly do not follow the same internal flow, even if the end result for the user looks similar.

The same is true for the API structures. The *Computer Associates International, Inc. v. Altai, Inc.* (1992) case set the well-known "abstraction–filtration–comparison" test that courts still use to decide whether parts of software code are protectable. Using this test, similarities caused by functional requirements or industry standards are filtered out before comparing anything. Eclipse IDE gathers almost everything under `org.eclipse.debug.*` with clearly layered packages, while IntelliJ IDEA spreads its older debugger under `com.intellij.debugger.*` and its newer debugger under `com.intellij.xdebugger.*`. Their organizational choices, naming conventions, and layering differ drastically. After filtering these differences, there is essentially nothing left that appears similar enough to raise concerns.

Finally, reviewing the source code around line breakpoint implementations also supports low risk. IntelliJ IDEA's breakpoint code is straightforward and Java-centric, while Eclipse IDE uses a more generalized, template-like style that integrates with the rest of the Eclipse platform. Both rely on the Java Debug Interface (JDI), which is part of the publicly available Java specification. Under the C*omputer Associates International, Inc. v. Altai, Inc.* (1992) ruling, any similarity coming from JDI usage would be filtered out because it's not protectable. What remains, the "original expression," looks different in both IDEs.

The legal precedents reinforce our technical conclusion: IntelliJ IDEA and Eclipse IDE do not demonstrate infringement in any area where intellectual property protection would apply.

## References


Apple Inc. v. Samsung Elec. Co., Ltd., No. 12-1507 (Fed. Cir. 2012)

https://law.justia.com/cases/federal/appellate-courts/cafc/12-1507/12-1507-2012-10-11.html

Computer Associates Int'l, Inc. v. Altai, Inc., 982 F.2d 693 (2d Cir. 1992).

https://law.justia.com/cases/federal/appellate-courts/F2/982/693/137252/.

Eclipse Foundation. (n.d.). *Eclipse platform*. GitHub.

https://github.com/eclipse-platform/eclipse.platform.

Eclipse Foundation. (n.d.). *Eclipse platform debugger*. GitHub.

https://github.com/eclipse-platform/eclipse.platform/tree/master/debug.

JetBrains. (n.d.). *IntelliJ IDEA Community Edition*. GitHub.

https://github.com/JetBrains/intellij-community.

JetBrains. (n.d.). *IntelliJ IDEA debugger*. GitHub.

https://github.com/JetBrains/intellij-community/tree/master/java/debugger.

Johnson Controls, Inc. v. Phoenix Control Systems, Inc., 886 F.2d 1173 (9th Cir. 1989).

https://law.justia.com/cases/federal/appellate-courts/F2/886/1173/19079/.

Lotus Development Corp. v. Borland Intern., Inc., 788 F. Supp. 78 (D. Mass. 1992).

https://law.justia.com/cases/federal/district-courts/FSupp/788/78/1599920/